Page No.
Date

Electric Vehicle Charging Scheduling

~~schedule~~ EV charging ~~device~~ based on charging demand and electricity prices

Objective:- Min charging cost and peak grid load.

Problem Statement

The objective is to develop a Genetic Algorithm (GA) to optimize Electric Vehicles (EV) charging schedules based on charging demand and electricity prices. The algorithm aims to minimize the total charging cost and peak grid load while satisfying EV charging requirements and grid capacity constraints

Pseudocode :-

Algorithm : Genetic Algorithm for EV charging scheduling

Input: EV charging demand, electricity prices, gridload, POP_SIZE, CROSSOVER_RATE, MUTATION_RATE, MAX_GEN

Output - Best charging schedule.

Date

1 Start
2 Define Fitness (schedule)
. Calculate total charging cost
. calculate peak grid load
. calculate objective = w1 × charging cost
+ w2 × peak grid load
. Return fitness = 1 / (1 + objective)

3 population ← generate random charging schedules (POP_SIZE)
4 for generation = 1 to MAX_GEN DO
5 Evaluate fitness of each schedule
6 Update best_solution if a better solution is found
7 new_population ← empty
8 while size(new_population) < POP_SIZE do
9 parent1, parent2 ← SELECT (population)
10 IF random() < CROSS OVER RATE THEN
11 child1, child2 ← (parent1, parent2)
end if
12 mutate (child1), mutate (child2) with
13 MUTATION RATE
12 ELSE
13 child1, child2 ← CROSSOVER (parent1, parent
14 END IF
15 mutate (child1), mutate (child2) with MUTATION_RATE
16 check charging demand and grid constraints.
Repair invalid schedules if necessary
17 Add child1, child2 to new population
18 End while

Page No:
Date:

19 population ← new_population
20 END FOR
21 Return best_solution
22 ~~STOP~~

Output:

Starting genetic algorithm optimization problem
Generation 001 | Best strategy Inverse | 0.018605
~~Generation 020~~
Inputs & Outputs overview
Target EV demand : 50.0 kWh | max Hourly Rate: 7.0 kW

HR	IN: Price	IN: Base	Out: EV charge	Total cost
00	$0.10	30	0.00	30.00
01	$0.10	25	2.48	27.48
02	$0.10	20	3.97	23.97
03	$0.10	20	0.00	20.00
04	$0.10	22	1.80	23.80
05	$0.12	35	3.94	38.94
06	$0.15	50	4.07	54.07
07	$0.12	65	2.73	67.73
08	$0.12	70	4.17	74.17
09	$0.12	75	2.67	77.67
10	$0.12	72	1.36	73.36
11	$0.12	70	0.19	70.19
12	$0.15	68	3.20	72.20
13	$0.15	70	2.41	72.41
14	$0.18	75	0.00	85.85
15	$0.22	85	0.85	93.00
16	$0.35	95	0.00	98.00

[1]
✓ 0s

```python
import random
import numpy as np

# --- 1. FIXED INPUT DATASET ---
PRICES = np.array([0.1, 0.1, 0.1, 0.1, 0.1, 0.12, 0.15, 0.15, 0.12, 0.12, 0.12, 0.12, 0.15, 0.
GRID_LOAD = np.array([30, 25, 20, 20, 22, 35, 50, 65, 70, 75, 72, 70, 68, 70, 75, 85, 95, 98,
DEMAND = 50.0  # Total kWh needed
MAX_RATE = 7.0 # Max kW per hour

# GA Hyperparameters
POP_SIZE, MAX_GEN, CR, MR = 40, 50, 0.8, 0.1

def fitness(sch):
    # Lower cost and lower peak load = Better fitness
    return 1 / (1 + (0.5 * np.sum(sch * PRICES) + 0.5 * np.max(GRID_LOAD + sch)))

def repair(sch):
    sch = np.clip(sch, 0, MAX_RATE)
    tot = np.sum(sch)
    if tot > 0:
        sch = np.clip((sch / tot) * DEMAND, 0, MAX_RATE)
    return sch
```

```python
# --- 2. GENETIC ALGORITHM LOOP ---
pop = [repair(np.random.uniform(0, MAX_RATE, 24)) for _ in range(POP_SIZE)]
best_sol = pop[0]

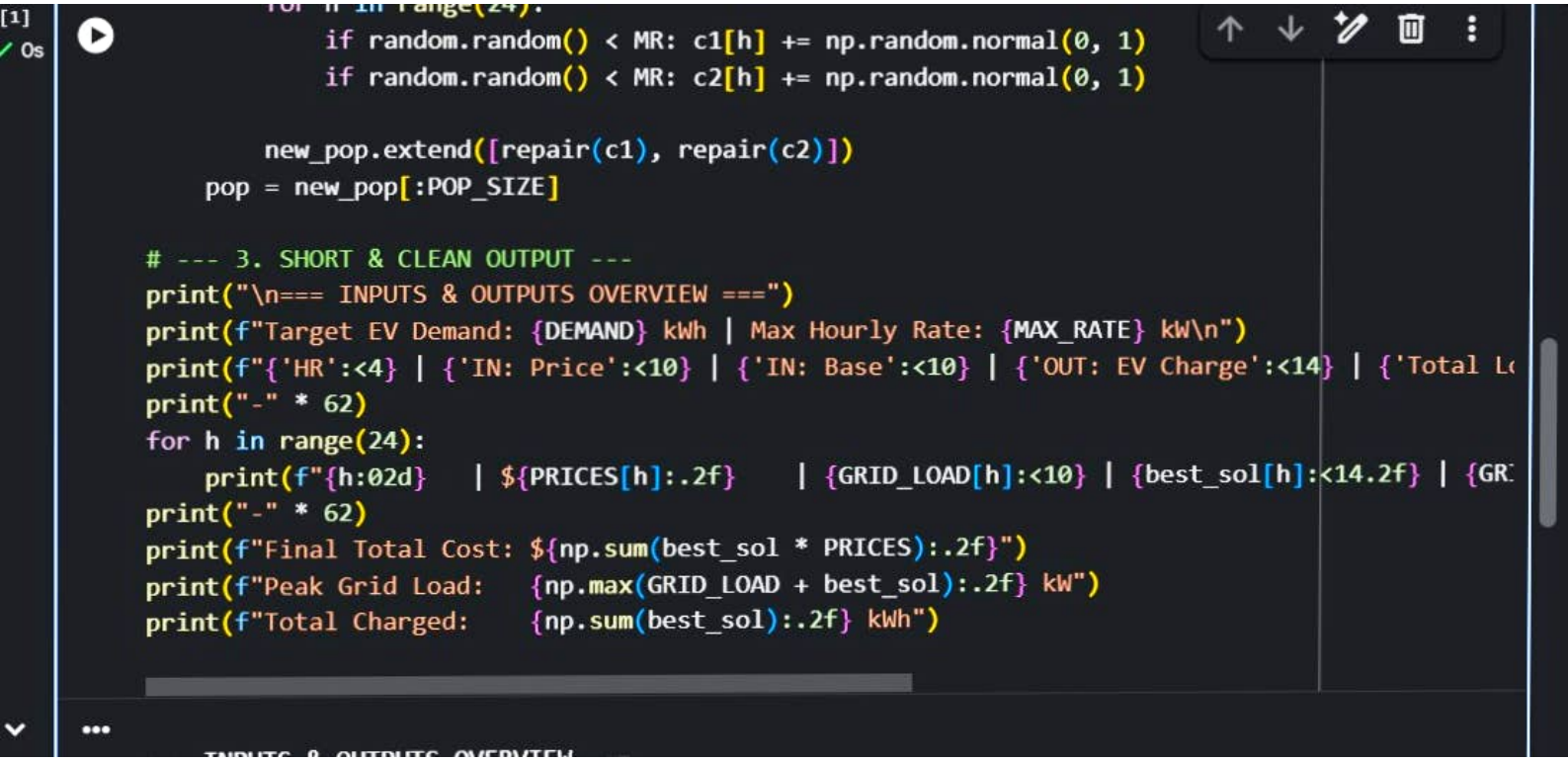
for gen in range(MAX_GEN):
    for s in pop:
        if fitness(s) > fitness(best_sol):
            best_sol = np.copy(s)

    new_pop = []
    while len(new_pop) < POP_SIZE:
        p1 = random.choice(pop) if fitness(random.choice(pop)) > fitness(random.choice(pop)) 
        p2 = random.choice(pop)

        c1, c2 = np.copy(p1), np.copy(p2)
        if random.random() < CR:
            pt = random.randint(1, 23)
            c1, c2 = np.concatenate((p1[:pt], p2[pt:])), np.concatenate((p2[:pt], p1[pt:]))

        for h in range(24):
            if random.random() < MR: c1[h] += np.random.normal(0, 1)
            if random.random() < MR: c2[h] += np.random.normal(0, 1)

        new_pop.extend([repair(c1), repair(c2)])
```



```
            if random.random() < MR: c1[h] += np.random.normal(0, 1)
            if random.random() < MR: c2[h] += np.random.normal(0, 1)

        new_pop.extend([repair(c1), repair(c2)])
    pop = new_pop[:POP_SIZE]

# --- 3. SHORT & CLEAN OUTPUT ---
print("\n=== INPUTS & OUTPUTS OVERVIEW ===")
print(f"Target EV Demand: {DEMAND} kWh | Max Hourly Rate: {MAX_RATE} kW\n")
print(f"{'HR':<4} | {'IN: Price':<10} | {'IN: Base':<10} | {'OUT: EV Charge':<14} | {'Total L
print("-" * 62)
for h in range(24):
    print(f"{h:02d}   | ${PRICES[h]:.2f}    | {GRID_LOAD[h]:<10} | {best_sol[h]:<14.2f} | {GR
print("-" * 62)
print(f"Final Total Cost: ${np.sum(best_sol * PRICES):.2f}")
print(f"Peak Grid Load:   {np.max(GRID_LOAD + best_sol):.2f} kW")
print(f"Total Charged:    {np.sum(best_sol):.2f} kWh")
```

```
=== INPUTS & OUTPUTS OVERVIEW ===
Target EV Demand: 50.0 kWh | Max Hourly Rate: 7.0 kW

HR   | IN: Price  | IN: Base   | OUT: EV Charge | Total Load
-------------------------------------------------------------
00   | $0.10      | 30         | 0.00           | 30.00
01   | $0.10      | 25         | 3.85           | 28.85
02   | $0.10      | 20         | 2.72           | 22.72
03   | $0.10      | 20         | 5.19           | 25.19
04   | $0.10      | 22         | 2.29           | 24.29
05   | $0.12      | 35         | 2.31           | 37.31
06   | $0.15      | 50         | 0.00           | 50.00
07   | $0.15      | 65         | 0.00           | 65.00
08   | $0.12      | 70         | 3.28           | 73.28
09   | $0.12      | 75         | 1.86           | 76.86
10   | $0.12      | 72         | 6.03           | 78.03
11   | $0.12      | 70         | 2.62           | 72.62
12   | $0.15      | 68         | 3.66           | 71.66
13   | $0.15      | 70         | 2.88           | 72.88
14   | $0.18      | 75         | 0.00           | 75.00
15   | $0.22      | 85         | 2.49           | 87.49
16   | $0.35      | 95         | 1.58           | 96.58
17   | $0.40      | 98         | 0.03           | 98.03
18   | $0.38      | 92         | 0.00           | 92.00
19   | $0.30      | 80         | 2.07           | 82.07
```

```
09   | $0.12    | 75          | 1.86           | 76.86
10   | $0.12    | 72          | 6.03           | 78.03
11   | $0.12    | 70          | 2.62           | 72.62
12   | $0.15    | 68          | 3.66           | 71.66
13   | $0.15    | 70          | 2.88           | 72.88
14   | $0.18    | 75          | 0.00           | 75.00
15   | $0.22    | 85          | 2.49           | 87.49
16   | $0.35    | 95          | 1.58           | 96.58
17   | $0.40    | 98          | 0.03           | 98.03
18   | $0.38    | 92          | 0.00           | 92.00
19   | $0.30    | 80          | 2.07           | 82.07
20   | $0.25    | 70          | 1.29           | 71.29
21   | $0.15    | 55          | 2.54           | 57.54
22   | $0.12    | 45          | 0.00           | 45.00
23   | $0.10    | 35          | 3.32           | 38.32
-------------------------------------------------------------
Final Total Cost: $7.09
Peak Grid Load:   98.03 kW
Total Charged:    50.00 kWh
```